

MADAPORC

Dossier de demarrage du code

Spring Boot MVC - Controllers, Services, Repositories, DTO, vues, references Figma et base de donnees

Element	Detail
Projet	MADAPORC - Application web de gestion d elevage porcin
Architecture cible	Java / Spring Boot MVC / JPA / Thymeleaf ou JSP
Style de code	@Controller + Model + @ModelAttribute + @RequestParam + @PathVariable + Service + Repository
Version	Document complet de demarrage - 2026-06-09
Important	Les rapports sont des pages calculees, pas des tables. La table journal_actions est supprimee. Le champ created_by est garde dans les tables importantes.

1. Objectif du document

Ce document transforme le cahier des charges et les ecrans Figma en plan de codage concret pour Spring Boot MVC. Il indique quoi creer: controllers, services, repositories, DTO, vues, validations, tables et colonnes.

2. Conventions de developpement

Le projet suit le style deja utilise: un Controller retourne une vue String, les donnees sont envoyees avec Model, les formulaires sont recus avec @ModelAttribute, les parametres de page avec @RequestParam ou @PathVariable, la logique est dans les Services et les acces aux donnees dans les Repositories JPA.

Element	Detail
Package racine conseille	com.madaporc
Controllers	com.madaporc.controller
Services	com.madaporc.service
Repositories	com.madaporc.repository
Entities JPA	com.madaporc.model
DTO	com.madaporc.DTO
Vues	src/main/resources/templates ou WEB-INF/views selon Thymeleaf/JSP
Retour Service	String erreur ou null pour les actions de formulaire, comme dans votre exemple
Transactions	@Transactional sur les methodes qui ajoutent/modifient plusieurs tables

3. Exemple de style a respecter

Exemple generique inspire de votre style de code:

```
@Controller
public class LotPorcController {
    private final LotPorcService lotPorcService;

    public LotPorcController(LotPorcService lotPorcService) {
        this.lotPorcService = lotPorcService;
    }

    @GetMapping("/lots")
    public String listLots(@RequestParam(value = "code", required = false) String code,
        @RequestParam(value = "raceId", required = false) Long raceId,
        @RequestParam(value = "statutId", required = false) Long statutId,
        Model model) {
        model.addAttribute("lots", lotPorcService.rechercherLots(code, raceId, statutId));
        model.addAttribute("races", lotPorcService.findAllRaces());
        model.addAttribute("statuts", lotPorcService.findAllStatutsLots());
        return "lots/listeLots";
    }

    @PostMapping("/lots/save")
    public String saveLot(@ModelAttribute LotPorcDTO dto, Model model, HttpSession session) {
        Long userId = (Long) session.getAttribute("userId");
        String erreur = dto.getId() == null
            ? lotPorcService.creer(dto, userId)
            : lotPorcService.modifier(dto.getId(), dto);
        model.addAttribute("message", erreur == null ? "Lot enregistre avec succes." : erreur);
        lotPorcService.prepareLotFormModel(model, dto.getId(), dto.getTypeEntree());
        return "lots/formLot";
    }
}
```

4. Fonctions generales du systeme

Element	Detail
Controller	Dans chaque controller: verifier la session si la page est protegee Utiliser Model pour les listes de reference Utiliser @ModelAttribute pour les DTO de formulaire
Service	Utilisateur getUtilisateurConnecte(HttpSession session) boolean verifierUtilisateurConnecte(HttpSession session) String verifierPermission(Long utilisateurId, String codePermission) String validerDonneesFormulaire(Object dto) String genererCodeUnique(String prefixe) List<T> paginer(List<T> liste, int page, int taille) String archiverElement(Long id, String tableLogique)
Repository	UtilisateurRepository.findById(Long id) PermissionRepository.findByCode(String code) RolePermissionRepository.findByRoleId(Long roleId)
DTO / Formulaire	Aucun DTO unique. Fonctions transversales utilisees par plusieurs modules.
Vue retournee	Toutes les vues protegees

5. Liste globale des controllers et routes

AuthController

- /login
- /logout

DashboardController

- /dashboard

LotPorcController

- /lots
- /lots/form
- /lots/save
- /lots/{id}

MouvementLotController

- /lots/{id}/mouvements
- /lots/mouvements/save

PeseeLotController

- /lots/{id}/pesees
- /lots/pesees/save

ReproducteurController

- /reproducteurs
- /reproducteurs/form
- /reproducteurs/save
- /reproducteurs/{id}

CycleProductionController

- /cycles
- /cycles/form
- /cycles/save

EvenementReproductionController

- /reproduction/evenements
- /reproduction/evenements/form

- /reproduction/evenements/save

ClientController

- /clients
- /clients/form
- /clients/save
- /clients/{id}

VenteController

- /ventes
- /ventes/form
- /ventes/save
- /ventes/valider/{id}
- /ventes/annuler/{id}

PaielementFactureController

- /factures/{ventelId}
- /paiements/save
- /factures/generer
- /factures/pdf/{ventelId}

DepenseController

- /depenses
- /depenses/form
- /depenses/save

IngredientController

- /ingredients
- /ingredients/form
- /ingredients/save

MouvementStockController

- /stocks/mouvements
- /stocks/mouvements/form
- /stocks/mouvements/save

MelangeController

- /melanges
- /melanges/form
- /melanges/save

DistributionAlimentController

- /distributions
- /distributions/form
- /distributions/save

VaccinController

- /vaccins
- /vaccins/form
- /vaccins/save

VaccinationController

- /vaccinations
- /vaccinations/form
- /vaccinations/save

SuiviSanitaireController

- /sante/suivis
- /sante/suivis/form
- /sante/suivis/save

EmployeController

- /employes
- /employes/form
- /employes/save

PresenceController

- /presences
- /presences/save
- /presences/pointer

SalaireController

- /salaires
- /salaires/generer
- /salaires/save
- /salaires/payer/{id}
- /salaires/fiche/{id}

RapportController

- /rapports
- /rapports/export/pdf
- /rapports/export/excel

ProfilController

- /profil
- /profil/save
- /profil/password

UtilisateurController

- /utilisateurs
- /utilisateurs/form
- /utilisateurs/save
- /utilisateurs/desactiver/{id}
- /roles/permissions/save

6. Detail complet des pages, fonctions et parametres

8.1 Page de connexion

Element	Detail
Reference Figma	Connexion - MADAPORC / GestPorc
Vue Spring MVC	login

Affichage a prevoir

- Champ email
- Champ mot de passe
- Bouton Connexion
- Message d'erreur
- Redirection vers le tableau de bord

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/login") String showLogin(Model model) @PostMapping("/login") String login(@ModelAttribute LoginDTO dto, Model model, HttpSession session) @GetMapping("/logout") String logout(HttpSession session)
Service	String connecter(LoginDTO dto, HttpSession session) boolean verifierMotDePasse(String brut, String hash) Optional<Utilisateur> findByEmail(String email) boolean utilisateurActif(Utilisateur utilisateur) String redirectionSelonRole(Utilisateur utilisateur)
Repository	Optional<Utilisateur> findByEmail(String email) List<Permission> findPermissionsByRoleId(Long roleId)
DTO / Formulaire	LoginDTO(email:String, motDePasse:String)
Vue retournee	login

Tables utilisees

- utilisateurs
- roles
- statuts_utilisateur
- permissions
- role_permissions

Regles et validations

- Le mot de passe est hashe.
- Un utilisateur desactive ne peut pas se connecter.
- La session doit contenir userId, roleId et nom.

8.2 Page tableau de bord

Element	Detail
Reference Figma	Tableau de bord - MADAPORC / GestPorc
Vue Spring MVC	dashboard/index

Affichage a prévoir

- Lots actifs
- Total porcs
- Porcs vendables
- Reproducteurs
- Porcs malades
- Ventes du mois
- Depenses du mois
- Benefice net
- Alertes stock
- Vaccinations a venir
- Graphiques evolution lots/ventes/stocks

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/dashboard") String dashboard(Model model, HttpSession session)
Service	DashboardDTO getDashboard(LocalDate debut, LocalDate fin) long compterLotsActifs() int calculerTotalPorcsActifs() int calculerPorcsVendables() long compterReproducteursActifs() long compterCasSanitairesEnCours() BigDecimal calculerVentesMois(LocalDate mois) BigDecimal calculerDepensesMois(LocalDate mois) BigDecimal calculerBeneficeNet(LocalDate debut, LocalDate fin) List<Ingredient> listerStocksFaibles() List<Vaccination> listerVaccinationsAVenir(LocalDate dateLimite)
Repository	LotPorcRepository.countByStatutLotLibelle(String statut) CycleProductionRepository.sumNombreVendablesActifs()

	VenteRepository.sumMontantByDateBetween(LocalDateTime debut, LocalDateTime fin) DepenseRepository.sumMontantByDateBetween(LocalDate debut, LocalDate fin) IngredientRepository.findByStockActuelKgLessThanEqualSeuilMinKg() VaccinationRepository.findByDateRappelBetween(LocalDate debut, LocalDate fin)
DTO / Formulaire	DashboardDTO
Vue retournee	dashboard/index

Tables utilisees

- lots_porcs
- cycles_production
- reproducteurs
- suivis_sanitaires
- vaccinations
- ventes
- details_vente
- depenses
- ingredients
- presences

Regles et validations

- Les statistiques sont calculees depuis la base.
- Les lots termines/archives ne sont pas actifs.
- Benefice net = ventes - depenses.

8.3 Page liste des lots de porcs

Element	Detail
Reference Figma	Liste des Lots - MADAPORC / GestPorc
Vue Spring MVC	lots/listeLots

Affichage a prevoir

- Tableau des lots
- Code lot
- Race
- Effectif initial
- Effectif actuel
- Poids moyen
- Date naissance/achat
- Statut
- Recherche, filtres, pagination
- Actions voir/modifier/archiver

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/lots") String listLots(@RequestParam(required=false) String code, @RequestParam(required=false) Long raceId, @RequestParam(required=false) Long statutId, Model model) @PostMapping("/lots/archive/{id}") String archiver(@PathVariable Long id, Model model)
Service	List<LotPorc> rechercherLots(String code, Long raceId, Long statutId) Optional<LotPorc> findLot(Long id) String archiverLot(Long lotId) boolean verifierLotActif(Long lotId) void prepareLotListModel(Model model, String code, Long raceId, Long statutId)
Repository	List<LotPorc> findByCodeLotContainingIgnoreCase(String code) List<LotPorc> findByRaceIdAndStatutLotId(Long raceId, Long statutId) Optional<LotPorc> findById(Long id) boolean existsByCodeLot(String codeLot)
DTO / Formulaire	LotFilterDTO(code:String, raceId:Long, statutId:Long)
Vue retournee	lots/listeLots

Tables utilisees

- lots_porcs
- races
- statuts_lots

Regles et validations

- Chaque lot a un code unique.
- Un lot archive ne doit plus apparaitre dans les lots actifs.
- Le nombre actuel reste coherent avec les mouvements.

8.4 Page ajout / modification d un lot de porcs

Element	Detail
Reference Figma	Ajouter un Lot - Naissance / Ajouter un Lot - Achat - MADAPORC
Vue Spring MVC	lots/formLot

Affichage a prevoir

- Formulaire type entree: Naissance ou Achat
- Code lot
- Race
- Statut
- Nombre males/femelles
- Nombre initial et actuel
- Date naissance ou achat
- Prix achat total si achat
- Poids moyen initial
- Observation
- Boutons enregistrer/annuler

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/lots/form") String showForm(@RequestParam(required=false) Long id, @RequestParam(required=false) String typeEntree, Model model) @PostMapping("/lots/save") String saveLot(@ModelAttribute LotPorcDTO dto, Model model, HttpSession session)
Service	void prepareLotFormModel(Model model, Long id, String typeEntree) String creer(LotPorcDTO dto, Long utilisateurId) String modifier(Long id, LotPorcDTO dto) String verifierCodeUnique(String codeLot, Long idActuel) String validerDonneesLot(LotPorcDTO dto) LotPorc convertirDtoVersEntity(LotPorcDTO dto)
Repository	boolean existsByCodeLot(String codeLot) Optional<LotPorc> findById(Long id) List<Race> findAll() List<StatutLot> findAll()
DTO / Formulaire	LotPorcDTO(id:Long, typeEntree:String, codeLot:String, raceId:Long, statutLotId:Long, nombreMalesInitial:Integer, nombreFemellesInitial:Integer, nombreInitial:Integer, nombreActuel:Integer, dateNaissanceEstimee:LocalDate, dateAchat:LocalDate, prixAchatTotal:BigDecimal, poidsMoyenInitialKg:BigDecimal, observation:String)
Vue retournee	lots/formLot

Tables utilisees

- lots_porcs
- races
- statuts_lots
- utilisateurs

Regles et validations

- Nombre initial > 0.
- Nombre actuel >= 0.
- Date non future.
- Pour type Achat, dateAchat et prixAchatTotal sont obligatoires.
- Pour type Naissance, dateNaissanceEstimee est obligatoire.

8.5 Page detail d un lot de porcs

Element	Detail
Reference Figma	Detail du Lot - MADAPORC / GestPorc
Vue Spring MVC	lots/detailLot

Affichage a prévoir

- Informations generales
- Effectif actuel
- Age moyen
- Origine et entree
- Planning prevu
- Mortalite cumulee
- GMQ moyen
- Poids moyen actuel
- Evolution du poids
- Onglets: informations, mouvements, pesees, sante, alimentation, ventes

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/lots/{id}") String detailLot(@PathVariable Long id, Model model) @GetMapping("/lots/{id}/tab/{tab}") String detailLotTab(@PathVariable Long id, @PathVariable String tab, Model model)
Service	LotDetailDTO getDetailLot(Long lotId) List<MouvementLotPorc> getMouvements(Long lotId) List<PeseeLot> getPesees(Long lotId) List<SuiviSanitaire> getSuivisSanitaires(Long lotId) List<Vaccination> getVaccinations(Long lotId) List<DistributionAliment> getDistributions(Long lotId) List<DetailVente> getVentes(Long lotId) BigDecimal calculerTauxMortalite(Long lotId) BigDecimal calculerGMQMoyen(Long lotId)
Repository	LotPorcRepository.findById(Long id) MouvementLotPorcRepository.findByLotPorcOrderByDateMouvementDesc(LotPorc lot) PeseeLotRepository.findByLotPorcOrderByDatePeseeAsc(LotPorc lot) SuiviSanitaireRepository.findByLotPorc(LotPorc lot) VaccinationRepository.findByLotPorc(LotPorc lot) DistributionAlimentRepository.findByLotPorc(LotPorc lot) DetailVenteRepository.findByLotPorc(LotPorc lot)
DTO / Formulaire	LotDetailDTO
Vue retournee	lots/detailLot

Tables utilisees

- lots_porcs
- mouvements_lots_porcs
- pesees_lots
- suivis_sanitaires
- vaccinations
- distributions_aliment
- details_vente

Regles et validations

- Le detail doit permettre de comprendre l evolution complete du lot.
- Les mouvements expliquent les variations de l effectif actuel.

8.6 Page mouvements des lots

Element	Detail
Reference Figma	Detail du Lot - Onglet Mouvements / Liste des Lots - Actions
Vue Spring MVC	lots/mouvements

Affichage a prevoir

- Liste des mouvements
- Lot concerne
- Type mouvement
- Quantite
- Date
- Motif
- Responsable
- Bouton nouveau mouvement

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/lots/{id}/mouvements") String listMouvements(@PathVariable Long id, Model model) @PostMapping("/lots/mouvements/save") String saveMouvement(@ModelAttribute MouvementLotDTO dto, Model model, HttpSession session)
Service	String ajouterMouvement(MouvementLotDTO dto, Long utilisateurId) List<MouvementLotPorc> findMouvements(Long lotId) String verifierQuantiteDisponible(Long lotId, Integer quantite, String typeMouvement) void mettreAJourEffectifLot(Long lotId, Integer quantite, String typeMouvement) String validerMouvementLot(MouvementLotDTO dto)
Repository	MouvementLotPorcRepository.findByLotPorcOrderByDateMouvementD esc(LotPorc lot) TypeMouvementLotRepository.findAll() LotPorcRepository.findById(Long id)
DTO / Formulaire	MouvementLotDTO(id:Long, lotPorcId:Long, typeMouvementLotId:Long, quantite:Integer, dateMouvement:LocalDateTime, motif:String)
Vue retournee	lots/mouvements

Tables utilisees

- mouvements_lots_porcs
- type_mouvements_lots
- lots_porcs
- utilisateurs

Regles et validations

- Une entree augmente le nombre actuel.
- Une sortie/perde/deces/vente diminue le nombre actuel.
- Quantite > 0.
- Sortie <= nombre actuel.

8.7 Page suivi du poids des lots

Element	Detail
Reference Figma	Detail du Lot - Evolution du poids / Onglet Pesees
Vue Spring MVC	lots/pesees

Affichage a prevoir

- Liste des pesees
- Lot
- Date pesee
- Poids moyen

- Observation
- Graphique evolution
- Bouton ajouter pesee

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/lots/{id}/pesees") String listPesees(@PathVariable Long id, Model model) @PostMapping("/lots/pesees/save") String savePesee(@ModelAttribute PeseeLotDTO dto, Model model, HttpSession session)
Service	String ajouterPesee(PeseeLotDTO dto, Long utilisateurId) List<PeseeLot> findPesees(Long lotId) BigDecimal calculerEvolutionPoids(Long lotId) void mettreAJourPoidsMoyenActuel(Long lotId, BigDecimal poidsMoyenKg) String validerPesee(PeseeLotDTO dto)
Repository	PeseeLotRepository.findByLotPorcOrderByDatePeseeAsc(LotPorc lot) LotPorcRepository.findById(Long id)
DTO / Formulaire	PeseeLotDTO(id:Long, lotPorcId:Long, poidsMoyenKg:BigDecimal, datePesee:LocalDate, observation:String)
Vue retournee	lots/pesees

Tables utilisees

- pesees_lots
- lots_porcs
- utilisateurs

Regles et validations

- Pesee liee a un lot existant.
- Poids moyen > 0.
- Date pesee non future.

8.8 Page liste des reproducteurs

Element	Detail
Reference Figma	Liste des Reproducteurs - MADAPORC / GestPorc
Vue Spring MVC	reproducteurs/listeReproducteurs

Affichage a prévoir

- Liste des reproducteurs
- Code
- Nom
- Race
- Sexe
- Date naissance
- Poids
- Statut
- Recherche, filtres, pagination
- Actions ajouter/modifier/detail/archiver

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/reproducteurs") String listReproducteurs(@RequestParam(required=false) String motCle, @RequestParam(required=false) Long sexId, @RequestParam(required=false) Long statutId, Model model) @GetMapping("/reproducteurs/form") String showForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/reproducteurs/save") String save(@ModelAttribute ReproducteurDTO dto, Model model, HttpSession session) @PostMapping("/reproducteurs/archive/{id}") String archiver(@PathVariable Long id, Model model)
Service	List<Reproducteur> rechercherReproducteurs(String motCle, Long sexId, Long statutId)

	void prepareReproducteurFormModel(Model model, Long id) String creer(ReproducteurDTO dto, Long utilisateurId) String modifier(Long id, ReproducteurDTO dto) String archiverReproducteur(Long id) String verifierCodeUnique(String codeReproducteur, Long idActuel)
Repository	ReproducteurRepository.findByCodeReproducteurContainingIgnoreCaseOrNomContainingIgnoreCase(String code, String nom) ReproducteurRepository.existsByCodeReproducteur(String code) RaceRepository.findAll() SexeRepository.findAll() StatutReproducteurRepository.findAll()
DTO / Formulaire	ReproducteurDTO(id:Long, codeReproducteur:String, nom:String, raceId:Long, sexeId:Long, statutReproducteurId:Long, dateNaissance:LocalDate, dateArrivee:LocalDate, prixAchat:BigDecimal, poidsKg:BigDecimal, observation:String)
Vue retournee	reproducteurs/listeReproducteurs

Tables utilisees

- reproducteurs
- races
- sexes
- statuts_reproducteurs
- utilisateurs

Regles et validations

- Code reproducteur unique.
- Un reproducteur archive/mort/reforme n est pas actif.
- Poids > 0 si renseigne.

8.9 Page detail d un reproducteur

Element	Detail
Reference Figma	Detail du Reproducteur - MADAPORC / GestPorc
Vue Spring MVC	reproducteurs/detailReproducteur

Affichage a prévoir

- Informations generales
- Cycle de reproduction actuel
- Historique reproduction
- Historique sanitaire
- Vaccinations
- Bouton modifier

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/reproducteurs/{id}") String detail(@PathVariable Long id, Model model)
Service	ReproducteurDetailDTO getDetailReproducteur(Long id) List<EvenementReproduction> getEvenements(Long reproducteurId) List<SuiviSanitaire> getSuivisSanitaires(Long reproducteurId) List<Vaccination> getVaccinations(Long reproducteurId) Optional<EvenementReproduction> getCycleActuel(Long reproducteurId)
Repository	ReproducteurRepository.findById(Long id) EvenementReproductionRepository.findByIdByFemelleIdOrMaleIdOrderByDateEvenementDesc(Long femelleId, Long maleId) SuiviSanitaireRepository.findByReproducteur(Reproducteur reproducteur) VaccinationRepository.findByReproducteur(Reproducteur reproducteur)
DTO / Formulaire	ReproducteurDetailDTO
Vue retournee	reproducteurs/detailReproducteur

Tables utilisees

- reproducteurs

- suivis_sanitaires
- vaccinations
- evenements_reproduction
- statuts_reproducteurs

Regles et validations

- Le detail montre l historique sanitaire et reproductif.
- Les evenements doivent etre lies a des reproducteurs existants.

8.10 Page cycles de production

Element	Detail
Reference Figma	Cycles de Production - MADAPORC / GestPorc
Vue Spring MVC	production/cycles

Affichage a prévoir

- Cartes: cycles actifs, naissances du mois, taux de perte moyen
- Liste des cycles
- Code cycle
- Lot concerne
- Dates
- Naissances
- Pertes
- Vivants
- Vendables
- Observation
- Boutons filtrer/nouveau cycle

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/cycles") String listCycles(Model model) @GetMapping("/cycles/form") String showCycleForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/cycles/save") String saveCycle(@ModelAttribute CycleProductionDTO dto, Model model)
Service	List<CycleProduction> findAllCycles() String creer(CycleProductionDTO dto) String modifier(Long id, CycleProductionDTO dto) int calculerNombreVivants(Integer naissances, Integer pertes) int calculerNombreVendables(Long cycleId) BigDecimal calculerTauxPerte(Long cycleId) void cloturerCycle(Long cycleId, LocalDate dateFinReelle)
Repository	CycleProductionRepository.findAllByOrderByDateDebutDesc() CycleProductionRepository.existsByCodeCycle(String code) LotPorcRepository.findAll()
DTO / Formulaire	CycleProductionDTO(id:Long, codeCycle:String, lotPorcId:Long, dateDebut:LocalDate, dateFinPrevue:LocalDate, dateFinReelle:LocalDate, nombreNaissances:Integer, nombrePertes:Integer, nombreVivants:Integer, nombreVendables:Integer, statutCycle:String, observation:String)
Vue retournee	production/cycles

Tables utilisees

- cycles_production
- lots_porcs

Regles et validations

- Un cycle est lie a un lot.
- Vivants = naissances - pertes.
- Vendables <= vivants.
- Code cycle unique.

8.11 Page evenements de reproduction

Element	Detail
Reference Figma	Evenements de Reproduction - MADAPORC / GestPorc
Vue Spring MVC	reproduction/evenements

Affichage a prévoir

- Cartes: saillies semaine, gestations confirmees, mises bas du mois, alertes actives
- Chronologie detaillee
- Type evenement
- Femelle
- Male/lot
- Porcelets
- Observation
- Filtres, export

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/reproduction/evenements") String listEvenements(@RequestParam(required=false) Long typeId, Model model) @GetMapping("/reproduction/evenements/form") String showEvenementForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/reproduction/evenements/save") String saveEvenement(@ModelAttribute EvenementReproductionDTO dto, Model model, HttpSession session)
Service	List<EvenementReproduction> rechercherEvenements(Long typeId) String ajouter(EvenementReproductionDTO dto, Long utilisateurId) String modifier(Long id, EvenementReproductionDTO dto) int calculerPorceletsVivants(Integer nes, Integer morts) String creerLotApresMiseBas(EvenementReproductionDTO dto) void mettreAJourCycleApresMiseBas(Long evenementId)
Repository	EvenementReproductionRepository.findByIdEvenementReproductio nId(Long typeId) TypeEvenementReproductionRepository.findAll() ReproducteurRepository.findBySexeLibelle(String sexe) LotPorcRepository.findAll()
DTO / Formulaire	EvenementReproductionDTO(id:Long, femelleId:Long, maleId:Long, typeEvenementReproductionId:Long, lotPorcId:Long, dateEvenement:LocalDate, nombrePorceletsNes:Integer, nombrePorceletsMorts:Integer, nombrePorceletsVivants:Integer, observation:String)
Vue retournee	reproduction/evenements

Tables utilisees

- evenements_reproduction
- type_evenements_reproduction
- reproducteurs
- lots_porcs
- utilisateurs

Regles et validations

- Une mise bas est liee a une femelle.
- Vivants <= nes.
- Si creation de lot apres mise bas, code lot unique.

8.12 Page gestion des clients / acheteurs

Element	Detail
Reference Figma	Gestion des Clients - MADAPORC / GestPorc
Vue Spring MVC	commerce/clients

Affichage a prevoir

- Liste des clients
- Nom
- Type
- Contact
- Email
- Adresse
- Recherche, filtres
- Actions ajouter/modifier/detail

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/clients") String listClients(@RequestParam(required=false) String motCle, @RequestParam(required=false) String typeClient, Model model) @GetMapping("/clients/form") String showClientForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/clients/save") String saveClient(@ModelAttribute ClientDTO dto, Model model) @GetMapping("/clients/{id}") String detailClient(@PathVariable Long id, Model model)
Service	List<Client> rechercherClients(String motCle, String typeClient) String creer(ClientDTO dto) String modifier(Long id, ClientDTO dto) Optional<Client> findClient(Long id) List<Vente> getVentesClient(Long clientId) String validerClient(ClientDTO dto)
Repository	ClientRepository.findByNomContainingIgnoreCase(String nom) ClientRepository.findByTypeClient(String typeClient) VenteRepository.findByClientOrderByDateVenteDesc(Client client)
DTO / Formulaire	ClientDTO(id:Long, nom:String, typeClient:String, contact:String, email:String, adresse:String)
Vue retournee	commerce/clients

Tables utilisees

- clients
- ventes

Regles et validations

- Un client avec ventes ne doit pas etre supprime sans controle.
- Email valide si renseigne.

8.13 Page gestion des ventes

Element	Detail
Reference Figma	Gestion des Ventes - MADAPORC / GestPorc
Vue Spring MVC	commerce/ventes

Affichage a prevoir

- Cartes: chiffre affaires du mois, nombre ventes, ventes en attente
- Historique transactions
- Date
- Client
- Lot
- Quantite
- Poids total
- Montant
- Statut
- Ajouter/filtrer/exporter

Fonctions Spring Boot MVC a creer

Element	Detail
---------	--------

Controller	@GetMapping("/ventes") String listVentes(Model model) @GetMapping("/ventes/form") String showVenteForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/ventes/save") String saveVente(@ModelAttribute VenteDTO dto, Model model, HttpSession session) @PostMapping("/ventes/valider/{id}") String validerVente(@PathVariable Long id, Model model) @PostMapping("/ventes/annuler/{id}") String annulerVente(@PathVariable Long id, Model model)
Service	List<Vente> findAllVentes() void prepareVenteFormModel(Model model, Long id) String creer(VenteDTO dto, Long utilisateurId) String modifier(Long id, VenteDTO dto) BigDecimal calculerMontantDetail(Integer nbPorcs, BigDecimal poidsTotalKg, BigDecimal prixKg, BigDecimal prixUnitaire) String validerVente(Long venteld) String annulerVente(Long venteld) void mettreAJourLotApresVente(DetailVente detail) void creerMouvementLotApresVente(DetailVente detail, Long utilisateurId)
Repository	VenteRepository.findAllOrderByDateVenteDesc() DetailVenteRepository.findByVente(Vente vente) ClientRepository.findAll() LotPorcRepository.findByStatutLotLibelle(String statut) PaielementRepository.findByVente(Vente vente)
DTO / Formulaire	VenteDTO(id:Long, clientId:Long, dateVente:LocalDateTime, statutVente:String, observation:String, details:List<DetailVenteDTO>) ; DetailVenteDTO(lotPorcId:Long, nombrePorcsVendus:Integer, poidsTotalKg:BigDecimal, prixKg:BigDecimal, prixUnitaire:BigDecimal)
Vue retournee	commerce/ventes

Tables utilisees

- ventes
- details_vente
- clients
- lots_porcs
- paiements
- factures
- mouvements_lots_porcs
- utilisateurs

Regles et validations

- Montant calcule automatiquement.
- Nombre vendu <= nombre actuel du lot.
- Validation diminue l effectif du lot.
- Validation peut generer facture/recu.

8.14 Page paiements et factures

Element	Detail
Reference Figma	Paiements et Factures - MADAPORC / GestPorc
Vue Spring MVC	commerce/factureDetail

Affichage a prevoir

- Detail facture
- Informations paiement
- Montant total
- Montant paye
- Reste a payer
- Mode paiement
- Reference
- Bouton generer facture PDF

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/factures/{venteld}") String detailFacture(@PathVariable Long venteld, Model model) @PostMapping("/paiements/save") String savePaierement(@ModelAttribute PaiementDTO dto, Model model) @PostMapping("/factures/generer") String genererFacture(@RequestParam Long venteld, Model model) @GetMapping("/factures/pdf/{venteld}") String exporterFacturePdf(@PathVariable Long venteld, Model model)
Service	FactureDTO getFacture(Long venteld) String enregistrerPaierement(PaiementDTO dto) BigDecimal calculerResteAPayer(Long venteld) String genererFacture(Long venteld) String genererNumeroFacture() boolean ventePayeeTotalement(Long venteld)
Repository	VenteRepository.findById(Long id) PaiementRepository.findByVente(Vente vente) FactureRepository.findByVente(Vente vente) FactureRepository.existsByNumeroFacture(String numero)
DTO / Formulaire	PaiementDTO(id:Long, venteld:Long, montant:BigDecimal, modePaierement:String, reference:String, datePaierement:LocalDateTime) ; FactureDTO
Vue retournee	commerce/factureDetail

Tables utilisees

- paiements
- factures
- ventes
- clients
- details_vente

Regles et validations

- Paiement lie a vente existante.
- Numero facture unique.
- Montant paye >= 0.
- Reste a payer = total vente - paiements.

8.15 Page depenses

Element	Detail
Reference Figma	Analyses & Rapports - Depenses / Module Commerce-Finance
Vue Spring MVC	finance/depenses

Affichage a prevoir

- Liste des depenses
- Date
- Categorie
- Libelle
- Montant
- Description
- Responsable
- Filtres periode/categorie

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/depenses") String listDepenses(@RequestParam(required=false) LocalDate debut, @RequestParam(required=false) LocalDate fin, @RequestParam(required=false) Long categorieId, Model model) @GetMapping("/depenses/form") String showDepenseForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/depenses/save") String saveDepense(@ModelAttribute DepenseDTO dto, Model model),

	HttpSession session)
Service	List<Depense> rechercherDepenses(LocalDate debut, LocalDate fin, Long categorieId) String creer(DepenseDTO dto, Long utilisateurId) String modifier(Long id, DepenseDTO dto) BigDecimal calculerTotalDepenses(LocalDate debut, LocalDate fin) String validerDepense(DepenseDTO dto) void prepareDepenseFormModel(Model model, Long id)
Repository	DepenseRepository.findByDateDepenseBetween(LocalDate debut, LocalDate fin) DepenseRepository.findByCategorieDepenseId(Long categorieId) CategorieDepenseRepository.findAll()
DTO / Formulaire	DepenseDTO(id:Long, categorieDepenseId:Long, libelle:String, montant:BigDecimal, dateDepense:LocalDate, description:String)
Vue retournee	finance/depenses

Tables utilisees

- depenses
- categories_depenses
- utilisateurs

Regles et validations

- Toute depense a une categorie.
- Montant > 0.
- Les depenses entrent dans le benefice net.

8.16 Page gestion des aliments et ingredients

Element	Detail
Reference Figma	Gestion des Ingredients - MADAPORC / GestPorc
Vue Spring MVC	ressources/ingredients

Affichage a prévoir

- Liste des ingredients
- Nom ingredient
- Prix par unite/kg
- Stock actuel
- Seuil minimum
- Unite
- Statut
- Actions ajouter/modifier/desactiver

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/ingredients") String listIngredients(@RequestParam(required=false) String motCle, Model model) @GetMapping("/ingredients/form") String showIngredientForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/ingredients/save") String saveIngredient(@ModelAttribute IngredientDTO dto, Model model)
Service	List<Ingredient> rechercherIngredients(String motCle) String creer(IngredientDTO dto) String modifier(Long id, IngredientDTO dto) String desactiverIngredient(Long id) List<Ingredient> listerStocksFaibles() BigDecimal calculerValeurStock(Long ingredientId) String validerIngredient(IngredientDTO dto)
Repository	IngredientRepository.findByLibelleContainingIgnoreCase(String libelle) IngredientRepository.findByActifTrue() IngredientRepository.findByStockActuelKgLessThanEqualSeuilMinKg()
DTO / Formulaire	IngredientDTO(id:Long, libelle:String, prixKg:BigDecimal, stockActuelKg:BigDecimal, seuilMinKg:BigDecimal, unite:String, actif:Boolean)
Vue retournee	ressources/ingredients

Tables utilisees

- ingredients

Regles et validations

- Alerte si stock <= seuil.
- Prix >= 0.
- Stock >= 0.

8.17 Page mouvements de stock alimentaire

Element	Detail
Reference Figma	Mouvements de Stock - MADAPORC / GestPorc
Vue Spring MVC	ressources/mouvementsStock

Affichage a prevoir

- Liste mouvements stock
- Date/heure
- Ingredient
- Type
- Quantite
- Prix total
- Raison/lot
- Responsable
- Filtres periode/type/ingredient
- Bouton nouveau mouvement

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/stocks/mouvements") String listMouvementsStock(@RequestParam(required=false) Long ingredientId, @RequestParam(required=false) Long typeId, Model model) @GetMapping("/stocks/mouvements/form") String showMouvementStockForm(Model model) @PostMapping("/stocks/mouvements/save") String saveMouvementStock(@ModelAttribute MouvementStockDTO dto, Model model, HttpSession session)
Service	List<MouvementStockAliment> rechercherMouvements(Long ingredientId, Long typeId, LocalDate debut, LocalDate fin) String ajouterMouvement(MouvementStockDTO dto, Long utilisateurId) String verifierStockAvantSortie(Long ingredientId, BigDecimal quantiteKg) void mettreAJourStockIngredient(Long ingredientId, BigDecimal quantiteKg, String typeMouvement) BigDecimal calculerPrixTotal(Long ingredientId, BigDecimal quantiteKg) String validerMouvementStock(MouvementStockDTO dto)
Repository	MouvementStockAlimentRepository.findByIngredientId(Long ingredientId) MouvementStockAlimentRepository.findByTypeMouvementStockId(Long typeId) IngredientRepository.findById(Long id) TypeMouvementStockRepository.findAll()
DTO / Formulaire	MouvementStockDTO(id:Long, ingredientId:Long, typeMouvementStockId:Long, quantiteKg:BigDecimal, prixTotal:BigDecimal, dateMouvement:LocalDateTime, motif:String)
Vue retournee	ressources/mouvementsStock

Tables utilisees

- mouvements_stock_aliment
- type_mouvements_stock
- ingredients
- utilisateurs

Regles et validations

- Entree augmente stock.
- Sortie/distribution diminue stock.
- Sortie <= stock disponible.

8.18 Page melanges alimentaires

Element	Detail
Reference Figma	Melanges Alimentaires / Feed Formulations - MADAPORC
Vue Spring MVC	ressources/melanges

Affichage a prevoir

- Liste des melanges
- Libelle
- Description
- Cout/kg
- Ingredients associes
- Pourcentage/quantite
- Details recette
- Boutons nouveau/export

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/melanges") String listMelanges(Model model) @GetMapping("/melanges/form") String showMelangeForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/melanges/save") String saveMelange(@ModelAttribute MelangeDTO dto, Model model)
Service	List<Melange> findAllMelanges() String creer(MelangeDTO dto) String modifier(Long id, MelangeDTO dto) String ajouterIngredientDansMelange(Long melangeld, Long ingredientId, BigDecimal quantiteKg, BigDecimal pourcentage) BigDecimal calculerCoutMelange(Long melangeld) String verifierTotalPourcentage(List<MelangeIngredientDTO> ingredients) MelangeDetailDTO getDetailMelange(Long id)
Repository	MelangeRepository.findAll() MelangeIngredientRepository.findByMelange(Melange melange) IngredientRepository.findAll() MelangeIngredientRepository.existsByMelangeldAndIngredientId(Long melangeld, Long ingredientId)
DTO / Formulaire	MelangeDTO(id:Long, libelle:String, description:String, ingredients:List<MelangeIngredientDTO>) ; MelangeIngredientDTO(ingredientId:Long, quantiteKg:BigDecimal, pourcentage:BigDecimal)
Vue retournee	ressources/melanges

Tables utilisees

- melanges
- melange_ingredients
- ingredients

Regles et validations

- Melange contient au moins un ingredient.
- Ingredient non repete dans le meme melange.
- Total pourcentage <= 100%.

8.19 Page distribution des aliments

Element	Detail
Reference Figma	Distribution des Aliments - MADAPORC / GestPorc
Vue Spring MVC	ressources/distributions

Affichage a prévoir

- Consommation journaliere
- Stock global restant
- Journal des distributions
- Lot concerne
- Melange distribue
- Date
- Quantite
- Cout total
- Observation/responsable

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/distributions") String listDistributions(Model model) @GetMapping("/distributions/form") String showDistributionForm(Model model) @PostMapping("/distributions/save") String saveDistribution(@ModelAttribute DistributionAlimentDTO dto, Model model, HttpSession session)
Service	List<DistributionAliment> findAllDistributions() String distribuer(DistributionAlimentDTO dto, Long utilisateurId) BigDecimal calculerCoutDistribution(Long melangeId, BigDecimal quantiteKg) String verifierStockPourDistribution(Long melangeId, BigDecimal quantiteKg) void diminuerStockIngredients(Long melangeId, BigDecimal quantiteDistribueeKg, Long utilisateurId) void creerMouvementsStockApresDistribution(DistributionAliment distribution) List<DistributionAliment> findByLot(Long lotId)
Repository	DistributionAlimentRepository.findAllByOrderByDateDistributionDesc() LotPorcRepository.findAll() MelangeRepository.findAll() MelangeIngredientRepository.findByMelange(Melange melange) IngredientRepository.findById(Long id)
DTO / Formulaire	DistributionAlimentDTO(id:Long, lotPorcId:Long, melangeId:Long, dateDistribution:LocalDate, quantiteKg:BigDecimal, observation:String)
Vue retournee	ressources/distributions

Tables utilisees

- distributions_aliment
- lots_porcs
- melanges
- melange_ingredients
- ingredients
- mouvements_stock_aliment
- utilisateurs

Regles et validations

- Quantite > 0.
- Distribution <= stock disponible.
- Stock diminue apres distribution.

8.20 Page gestion des vaccins

Element	Detail
Reference Figma	Gestion des Vaccins - MADAPORC / GestPorc
Vue Spring MVC	sante/vaccins

Affichage a prévoir

- Liste vaccins

- Nom vaccin
- Description
- Prix unite
- Delai rappel
- Statut
- Actions ajouter/modifier/desactiver
- Cartes vaccins actifs/budget/rappels critiques

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/vaccins") String listVaccins(Model model) @GetMapping("/vaccins/form") String showVaccinForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/vaccins/save") String saveVaccin(@ModelAttribute VaccinDTO dto, Model model)
Service	List<Vaccin> findAllVaccins() String creer(VaccinDTO dto) String modifier(Long id, VaccinDTO dto) String desactiverVaccin(Long id) long compterVaccinsActifs() BigDecimal calculerBudgetVaccins() long compterRappelsCritiques(LocalDate dateLimite)
Repository	VaccinRepository.findAll() VaccinRepository.findByActifTrue() VaccinationRepository.countByDateRappelBefore(LocalDate dateLimite)
DTO / Formulaire	VaccinDTO(id:Long, libelle:String, prix:BigDecimal, delaiRappelJours:Integer, description:String, actif:Boolean)
Vue retournee	sante/vaccins

Tables utilisees

- vaccins

Regles et validations

- Un vaccin peut etre utilise dans plusieurs vaccinations.
- Vaccin deja utilise: desactivation au lieu de suppression definitive.

8.21 Page vaccination / historique des vaccinations

Element	Detail
Reference Figma	Journal des Vaccinations - MADAPORC / GestPorc
Vue Spring MVC	sante/vaccinations

Affichage a prévoir

- Historique vaccinations
- Filtre lot/reproducteur
- Type cheptel
- Periode
- Vaccin utilise
- Date vaccination
- Date rappel
- Dose
- Observation
- Responsable
- Bouton enregistrer vaccination

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/vaccinations") String listVaccinations(@RequestParam(required=false) Long lotId, @RequestParam(required=false) Long reproducteurId, Model model) @GetMapping("/vaccinations/form") String

	showVaccinationForm(Model model) @PostMapping("/vaccinations/save") String saveVaccination(@ModelAttribute VaccinationDTO dto, Model model, HttpSession session)
Service	List<Vaccination> rechercherVaccinations(Long lotId, Long reproducteurId, LocalDate debut, LocalDate fin) String ajouter(VaccinationDTO dto, Long utilisateurId) LocalDate calculerDateRappel(Long vaccinId, LocalDate dateVaccination) String verifierCibleVaccination(Long lotId, Long reproducteurId) List<Vaccination> listerVaccinationsAVenir(LocalDate dateLimite)
Repository	VaccinationRepository.findByLotPorc(LotPorc lot) VaccinationRepository.findByReproducteur(Reproducteur reproducteur) VaccinationRepository.findByDateRappelBetween(LocalDate debut, LocalDate fin) VaccinRepository.findAll() LotPorcRepository.findAll() ReproducteurRepository.findAll()
DTO / Formulaire	VaccinationDTO(id:Long, lotPorcId:Long, reproducteurId:Long, vaccinId:Long, dateVaccination:LocalDate, dateRappel:LocalDate, dose:String, observation:String)
Vue retournee	sante/vaccinations

Tables utilisees

- vaccinations
- vaccins
- lots_porcs
- reproducteurs
- utilisateurs

Regles et validations

- Vaccination liee soit a un lot soit a un reproducteur.
- Vaccin existant obligatoire.
- Date rappel calculee depuis delaiRappelJours si vide.

8.22 Page suivi des maladies et traitements

Element	Detail
Reference Figma	Suivi Sanitaire - MADAPORC / GestPorc
Vue Spring MVC	sante/suivisSanitaires

Affichage a prévoir

- Cartes: cas en traitement, alertes critiques, taux guerison
- Registre recent
- Ref/lot
- Sujets
- Diagnostic initial
- Pathologie
- Protocole/traitement
- Statut
- Echeance
- Bouton nouveau cas/exporter

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/sante/suivis") String listSuivis(@RequestParam(required=false) Long statutId, Model model) @GetMapping("/sante/suivis/form") String showSuiviForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/sante/suivis/save") String saveSuivi(@ModelAttribute SuiviSanitaireDTO dto, Model model, HttpSession session) @PostMapping("/sante/suivis/statut") String changerStatut(@RequestParam Long suivId, @RequestParam Long statutId, Model model)

Service	List<SuiviSanitaire> rechercherSuivis(Long statutId, LocalDate debut, LocalDate fin) String ajouter(SuiviSanitaireDTO dto, Long utilisateurId) String modifier(Long id, SuiviSanitaireDTO dto) String modifierStatut(Long suivId, Long statutId) BigDecimal calculerTauxGuerison(LocalDate debut, LocalDate fin) BigDecimal calculerTauxMortalite(LocalDate debut, LocalDate fin) List<SuiviSanitaire> listerAlertesSanitaires() String verifierNombreMalades(Long lotId, Integer nombreMalades)
Repository	SuiviSanitaireRepository.findByStatutSuiviSanitaireId(Long statutId) MaladieRepository.findAll() TraitementRepository.findAll() StatutSuiviSanitaireRepository.findAll() LotPorcRepository.findAll() ReproducteurRepository.findAll()
DTO / Formulaire	SuiviSanitaireDTO(id:Long, lotPorcId:Long, reproducteurId:Long, nombrePorcsMalades:Integer, maladieId:Long, traitementId:Long, statutSuiviSanitaireId:Long, dateDiagnostic:LocalDate, dateGuerisonPrevue:LocalDate, dateGuerisonReelle:LocalDate, symptomes:String, diagnostic:String, observation:String)
Vue retournee	sante/suivisSanitaires

Tables utilisees

- suivis_sanitaires
- maladies
- traitements
- statuts_suivi_sanitaire
- lots_porcs
- reproducteurs
- utilisateurs

Regles et validations

- Suivi concerne soit un lot soit un reproducteur.
- Date diagnostic obligatoire.
- Nombre malades <= nombre actuel du lot.

8.23 Page employes

Element	Detail
Reference Figma	Gestion des Employes - MADAPORC / GestPorc
Vue Spring MVC	personnel/employes

Affichage a prévoir

- Liste employes
- Nom/prenom
- Contact
- Poste
- Salaire de base
- Statut
- Recherche/filtres
- Actions ajouter/modifier/detail

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/employes") String listEmployes(@RequestParam(required=false) String motCle, Model model) @GetMapping("/employes/form") String showEmployeForm(@RequestParam(required=false) Long id, Model model) @PostMapping("/employes/save") String saveEmploye(@ModelAttribute EmployeDTO dto, Model model)
Service	List<Employe> rechercherEmployes(String motCle, Long posteld, Long statutId) String creer(EmployeDTO dto)

	String modifier(Long id, EmployeeDTO dto) String archiverEmploye(Long id) BigDecimal getSalaireBase(Long employeId) void prepareEmployeeFormModel(Model model, Long id)
Repository	EmployeeRepository.findByNomContainingIgnoreCaseOrPrenomContainingIgnoreCase(String nom, String prenom) PosteEmployeeRepository.findAll() StatutEmployeeRepository.findAll()
DTO / Formulaire	EmployeeDTO(id:Long, nom:String, prenom:String, contact:String, adresse:String, posteEmployeId:Long, statutEmployeId:Long, dateEmbauche:LocalDate, salaireBase:BigDecimal)
Vue retournee	personnel/employees

Tables utilisees

- employes
- postes_employes
- statuts_employes

Regles et validations

- Employe actif/suspendu/parti.
- Employe inactif non utilise dans nouvelles operations.

8.24 Page presence / pointage

Element	Detail
Reference Figma	Presence et Pointage - MADAPORC / GestPorc
Vue Spring MVC	personnel/presences

Affichage a prévoir

- Effectif total
- Presents aujourd'hui
- Absences/retards
- Date
- Employe
- Arrivee
- Depart
- Statut
- Observation
- Bouton pointer/exporter

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/presences") String listPresences(@RequestParam(required=false) LocalDate date, Model model) @PostMapping("/presences/save") String savePresence(@ModelAttribute PresenceDTO dto, Model model) @PostMapping("/presences/pointer") String pointer(@RequestParam Long employeId, @RequestParam String statut, Model model)
Service	List<Presence> findPresences(LocalDate date) String enregistrerPresence(PresenceDTO dto) String pointerEmploye(Long employeId, String statut, LocalTime heure) boolean verifierPresenceDuJour(Long employeId, LocalDate date) long calculerNombreJoursPresence(Long employeId, int mois, int annee) long calculerAbsences(Long employeId, int mois, int annee) long calculerRetards(Long employeId, int mois, int annee)
Repository	PresenceRepository.findByDatePresence(LocalDate date) PresenceRepository.existsByEmployeIdAndDatePresence(Long employeId, LocalDate date) EmployeeRepository.findAll()
DTO / Formulaire	PresenceDTO(id:Long, employeId:Long, datePresence:LocalDate, statutPresence:String, heureArrivee:LocalTime, heureDepart:LocalTime, observation:String)
Vue retournee	personnel/presences

Tables utilisees

- presences
- employes

Regles et validations

- Un employe ne doit pas avoir deux pointages le meme jour.
- Date obligatoire.
- Heure depart apres heure arrivee si renseignees.

8.25 Page salaires employes

Element	Detail
Reference Figma	Gestion des Salaires - MADAPORC / GestPorc
Vue Spring MVC	personnel/salaires

Affichage a prevoir

- Mois courant
- Total masse salariale
- Primes versees
- Paiements en attente
- Registre salaires
- Base, prime, retenue, net a payer, statut, date paiement
- Boutons exporter/generer fiches

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/salaires") String listSalaires(@RequestParam(required=false) Integer mois, @RequestParam(required=false) Integer annee, Model model) @PostMapping("/salaires/generer") String genererSalaires(@RequestParam Integer mois, @RequestParam Integer annee, Model model) @PostMapping("/salaires/save") String saveSalaire(@ModelAttribute SalaireEmployeDTO dto, Model model) @PostMapping("/salaires/payer/{id}") String payerSalaire(@PathVariable Long id, Model model) @GetMapping("/salaires/fiche/{id}") String ficheSalaire(@PathVariable Long id, Model model)
Service	List<SalaireEmploye> findSalaires(Integer mois, Integer annee) String genererSalairesMois(Integer mois, Integer annee) String creerOuModifier(SalaireEmployeDTO dto) BigDecimal calculerPrime(Long employeId, Integer mois, Integer annee) BigDecimal calculerRetenue(Long employeId, Integer mois, Integer annee) BigDecimal calculerNet(BigDecimal base, BigDecimal prime, BigDecimal retenue) String marquerCommePaye(Long salaireId, LocalDate datePaiement) FicheSalaireDTO genererFicheSalaire(Long salaireId)
Repository	SalaireEmployeRepository.findByMoisAndAnnee(Integer mois, Integer annee) SalaireEmployeRepository.existsByEmployeIdAndMoisAndAnnee(Long employeId, Integer mois, Integer annee) EmployeRepository.findByStatutEmployeLibelle(String libelle) PresenceRepository.findByEmployeIdAndDatePresenceBetween(Long employeId, LocalDate debut, LocalDate fin)
DTO / Formulaire	SalaireEmployeDTO(id:Long, employeId:Long, mois:Integer, annee:Integer, montantBase:BigDecimal, prime:BigDecimal, retenue:BigDecimal, montantNet:BigDecimal, datePaiement:LocalDate, statutPaiement:String)
Vue retournee	personnel/salaires

Tables utilisees

- salaires_employes
- employes
- presences

Regles et validations

- Net = base + prime - retenue.
- Salaire lie a employe existant.
- Un salaire par employe par mois.

8.26 Page rapports

Element	Detail
Reference Figma	Analyses & Rapports - MADAPORC / GestPorc
Vue Spring MVC	rapports/index

Affichage a prevoir

- Rapport financier
- Rapport sanitaire
- Etat des stocks
- Presences
- Performance production
- Filtres periode/site
- Boutons Export PDF / Excel

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/rapports") String rapports(@RequestParam(required=false) LocalDate debut, @RequestParam(required=false) LocalDate fin, Model model) @GetMapping("/rapports/export/pdf") String exportPdf(@RequestParam LocalDate debut, @RequestParam LocalDate fin, Model model) @GetMapping("/rapports/export/excel") String exportExcel(@RequestParam LocalDate debut, @RequestParam LocalDate fin, Model model)
Service	RapportDTO genererRapportGlobal(LocalDate debut, LocalDate fin) RapportFinancierDTO genererRapportFinancier(LocalDate debut, LocalDate fin) RapportSanitaireDTO genererRapportSanitaire(LocalDate debut, LocalDate fin) RapportStockDTO genererRapportStock() RapportPresenceDTO genererRapportPresence(LocalDate debut, LocalDate fin) RapportProductionDTO genererRapportProduction(LocalDate debut, LocalDate fin)
Repository	VenteRepository.sumMontantByDateBetween(...) DepenseRepository.sumMontantByDateBetween(...) IngredientRepository.findAll() SuiviSanitaireRepository.findByDateDiagnosticBetween(...) PresenceRepository.findByDatePresenceBetween(...) CycleProductionRepository.findByDateDebutBetween(...)
DTO / Formulaire	RapportDTO, RapportFinancierDTO, RapportSanitaireDTO, RapportStockDTO, RapportPresenceDTO, RapportProductionDTO
Vue retournee	rapports/index

Tables utilisees

- ventes
- details_vente
- depenses
- categories_depenses
- ingredients
- mouvements_stock_aliment
- suivis_sanitaires
- vaccinations
- lots_porcs
- cycles_production
- reproducteurs
- presences

- employes

Regles et validations

- Rapports generes depuis les donnees existantes.
- Aucune table rapports_generes.
- Donnees validees seulement.

8.27 Page profil utilisateur

Element	Detail
Reference Figma	Profil Utilisateur - MADAPORC / GestPorc
Vue Spring MVC	settings/profil

Affichage a prevoir

- Nom
- Email
- Role
- Contact
- Localisation
- Bouton modifier profil
- Bouton changer mot de passe
- Preferences systeme

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/profil") String profil(HttpSession session, Model model) @PostMapping("/profil/save") String saveProfil(@ModelAttribute ProfilUtilisateurDTO dto, HttpSession session, Model model) @PostMapping("/profil/password") String changerMotDePasse(@ModelAttribute ChangerMotDePasseDTO dto, HttpSession session, Model model)
Service	Utilisateur getUtilisateurConnecte(HttpSession session) String modifierProfil(Long utilisateurId, ProfilUtilisateurDTO dto) String changerMotDePasse(Long utilisateurId, ChangerMotDePasseDTO dto) boolean verifierAncienMotDePasse(Long utilisateurId, String ancienMotDePasse) String hasherMotDePasse(String motDePasse)
Repository	UtilisateurRepository.findById(Long id) RoleRepository.findById(Long id)
DTO / Formulaire	ProfilUtilisateurDTO(nom:String, email:String) ; ChangerMotDePasseDTO(ancienMotDePasse:String, nouveauMotDePasse:String, confirmation:String)
Vue retournee	settings/profil

Tables utilisees

- utilisateurs
- roles

Regles et validations

- Utilisateur modifie seulement son profil.
- Nouveau mot de passe hashe.
- Confirmation mot de passe obligatoire.

8.28 Page gestion des utilisateurs et roles

Element	Detail
Reference Figma	Gestion des Utilisateurs - MADAPORC / GestPorc
Vue Spring MVC	settings/utilisateurs

Affichage a prevoir

- Cartes utilisateurs totaux/administrateurs/techniciens/comptes inactifs

- Annuaire personnel
- Nom/prenom/email/role/statut/actions
- Liste roles et permissions
- Bouton ajouter utilisateur

Fonctions Spring Boot MVC a creer

Element	Detail
Controller	@GetMapping("/utilisateurs") String listUtilisateurs(Model model, HttpSession session) @GetMapping("/utilisateurs/form") String showUtilisateurForm(@RequestParam(required=false) Long id, Model model, HttpSession session) @PostMapping("/utilisateurs/save") String saveUtilisateur(@ModelAttribute UtilisateurDTO dto, Model model, HttpSession session) @PostMapping("/utilisateurs/desactiver/{id}") String desactiver(@PathVariable Long id, Model model, HttpSession session) @PostMapping("/roles/permissions/save") String saveRolePermissions(@ModelAttribute RolePermissionDTO dto, Model model, HttpSession session)
Service	String verifierPermissionAdmin(Long utilisateurId) List<Utilisateur> findAllUtilisateurs() String creer(UtilisateurDTO dto) String modifier(Long id, UtilisateurDTO dto) String changerRole(Long utilisateurId, Long roleId) String desactiverUtilisateur(Long utilisateurId) String activerUtilisateur(Long utilisateurId) String affecterPermissions(RolePermissionDTO dto) Map<Role,List<Permission>> getRolesPermissions()
Repository	UtilisateurRepository.findAll() UtilisateurRepository.existsByEmail(String email) RoleRepository.findAll() PermissionRepository.findAll() RolePermissionRepository.deleteByRoleId(Long roleId) RolePermissionRepository.findByRoleId(Long roleId)
DTO / Formulaire	UtilisateurDTO(id:Long, nom:String, email:String, motDePasse:String, roleId:Long, statutUtilisateurId:Long) ; RolePermissionDTO(roleId:Long, permissionIds:List<Long>)
Vue retournee	settings/utilisateurs

Tables utilisees

- utilisateurs
- roles
- permissions
- role_permissions
- statuts_utilisateur

Regles et validations

- Seul administrateur gere utilisateurs et roles.
- Utilisateur desactive ne peut pas se connecter.
- Permissions limitent les pages sensibles.

7. Liste des DTO a creer

Element	Detail
Page de connexion	LoginDTO(email:String, motDePasse:String)
Page tableau de bord	DashboardDTO
Page liste des lots de porcs	LotFiltreDTO(code:String, raceId:Long, statutId:Long)
Page ajout / modification d un lot de porcs	LotPorcDTO(id:Long, typeEntree:String, codeLot:String, raceId:Long, statutLotId:Long, nombreMalesInitial:Integer, nombreFemellesInitial:Integer, nombreInitial:Integer, nombreActuel:Integer, dateNaissanceEstimee:LocalDate, dateAchat:LocalDate, prixAchatTotal:BigDecimal, poidsMoyenInitialKg:BigDecimal, observation:String)
Page detail d un lot de porcs	LotDetailDTO
Page mouvements des lots	MouvementLotDTO(id:Long, lotPorcId:Long, typeMouvementLotId:Long, quantite:Integer,

	dateMouvement:LocalDateTime, motif:String)
Page suivi du poids des lots	PeseeLotDTO(id:Long, lotPorcId:Long, poidsMoyenKg:BigDecimal, datePesee:LocalDate, observation:String)
Page liste des reproducteurs	ReproducteurDTO(id:Long, codeReproducteur:String, nom:String, raceId:Long, sexeId:Long, statutReproducteurId:Long, dateNaissance:LocalDate, dateArrivee:LocalDate, prixAchat:BigDecimal, poidsKg:BigDecimal, observation:String)
Page detail d un reproducteur	ReproducteurDetailDTO
Page cycles de production	CycleProductionDTO(id:Long, codeCycle:String, lotPorcId:Long, dateDebut:LocalDate, dateFinPrevue:LocalDate, dateFinReelle:LocalDate, nombreNaissances:Integer, nombrePertes:Integer, nombreVivants:Integer, nombreVendables:Integer, statutCycle:String, observation:String)
Page evenements de reproduction	EvenementReproductionDTO(id:Long, femelleId:Long, maleId:Long, typeEvenementReproductionId:Long, lotPorcId:Long, dateEvenement:LocalDate, nombrePorceletsNes:Integer, nombrePorceletsMorts:Integer, nombrePorceletsVivants:Integer, observation:String)
Page gestion des clients / acheteurs	ClientDTO(id:Long, nom:String, typeClient:String, contact:String, email:String, adresse:String)
Page gestion des ventes	VenteDTO(id:Long, clientId:Long, dateVente:LocalDateTime, statutVente:String, observation:String, details:List<DetailVenteDTO>) ; DetailVenteDTO(lotPorcId:Long, nombrePorcsVendus:Integer, poidsTotalKg:BigDecimal, prixKg:BigDecimal, prixUnitaire:BigDecimal)
Page paiements et factures	PaieementDTO(id:Long, ventId:Long, montant:BigDecimal, modePaieement:String, reference:String, datePaieement:LocalDateTime) ; FactureDTO
Page depenses	DepenseDTO(id:Long, categorieDepenseId:Long, libelle:String, montant:BigDecimal, dateDepense:LocalDate, description:String)
Page gestion des aliments et ingredients	IngredientDTO(id:Long, libelle:String, prixKg:BigDecimal, stockActuelKg:BigDecimal, seuilMinKg:BigDecimal, unite:String, actif:Boolean)
Page mouvements de stock alimentaire	MouvementStockDTO(id:Long, ingredientId:Long, typeMouvementStockId:Long, quantiteKg:BigDecimal, prixTotal:BigDecimal, dateMouvement:LocalDateTime, motif:String)
Page melanges alimentaires	MelangeDTO(id:Long, libelle:String, description:String, ingredients:List<MelangeIngredientDTO>) ; MelangeIngredientDTO(ingredientId:Long, quantiteKg:BigDecimal, pourcentage:BigDecimal)
Page distribution des aliments	DistributionAlimentDTO(id:Long, lotPorcId:Long, melangeId:Long, dateDistribution:LocalDate, quantiteKg:BigDecimal, observation:String)
Page gestion des vaccins	VaccinDTO(id:Long, libelle:String, prix:BigDecimal, delaiRappelJours:Integer, description:String, actif:Boolean)
Page vaccination / historique des vaccinations	VaccinationDTO(id:Long, lotPorcId:Long, reproducteurId:Long, vaccinId:Long, dateVaccination:LocalDate, dateRappel:LocalDate, dose:String, observation:String)
Page suivi des maladies et traitements	SuiviSanitaireDTO(id:Long, lotPorcId:Long, reproducteurId:Long, nombrePorcsMalades:Integer, maladieId:Long, traitementId:Long, statutSuiviSanitaireId:Long, dateDiagnostic:LocalDate, dateGuerisonPrevue:LocalDate, dateGuerisonReelle:LocalDate, symptomes:String, diagnostic:String, observation:String)
Page employes	EmployeDTO(id:Long, nom:String, prenom:String, contact:String, adresse:String, posteEmployeId:Long, statutEmployeId:Long, dateEmbauche:LocalDate, salaireBase:BigDecimal)
Page presence / pointage	PresenceDTO(id:Long, employeId:Long, datePresence:LocalDate, statutPresence:String, heureArrivee:LocalTime, heureDepart:LocalTime, observation:String)
Page salaires employes	SalaireEmployeDTO(id:Long, employeId:Long, mois:Integer, annee:Integer, montantBase:BigDecimal, prime:BigDecimal, retenue:BigDecimal, montantNet:BigDecimal, datePaieement:LocalDate, statutPaieement:String)
Page rapports	RapportDTO, RapportFinancierDTO, RapportSanitaireDTO, RapportStockDTO, RapportPresenceDTO, RapportProductionDTO
Page profil utilisateur	ProfilUtilisateurDTO(nom:String, email:String) ; ChangerMotDePasseDTO(ancienMotDePasse:String, nouveauMotDePasse:String, confirmation:String)
Page gestion des utilisateurs et roles	UtilisateurDTO(id:Long, nom:String, email:String, motDePasse:String, roleId:Long, statutUtilisateurId:Long) ; RolePermissionDTO(roleId:Long, permissionIds:List<Long>)

8. Entites JPA principales

- Utilisateur -> table utilisateurs
- Role -> table roles
- Permission -> table permissions
- StatutUtilisateur -> table statuts_utilisateur
- LotPorc -> table lots_porcs
- Race -> table races
- Sexe -> table sexes
- StatutLot -> table statuts_lots
- MouvementLotPorc -> table mouvements_lots_porcs
- TypeMouvementLot -> table type_mouvements_lots
- PeseeLot -> table pesees_lots
- Reproducteur -> table reproducteurs
- StatutReproducteur -> table statuts_reproducteurs
- CycleProduction -> table cycles_production
- EvenementReproduction -> table evenements_reproduction
- TypeEvenementReproduction -> table type_evenements_reproduction
- Maladie -> table maladies
- Traitement -> table traitements
- SuiviSanitaire -> table suivis_sanitaires
- StatutSuiviSanitaire -> table statuts_suivi_sanitaire
- Vaccin -> table vaccins
- Vaccination -> table vaccinations
- Ingredient -> table ingredients
- TypeMouvementStock -> table type_mouvements_stock
- MouvementStockAliment -> table mouvements_stock_aliment
- Melange -> table melanges
- MelangeIngredient -> table melange_ingredients
- RatioCroissance -> table ratios_croissance
- DistributionAliment -> table distributions_aliment
- Client -> table clients
- Vente -> table ventes
- DetailVente -> table details_vente
- Paiement -> table paiements
- Facture -> table factures
- CategorieDepense -> table categories_depenses
- Depense -> table depenses
- Transport -> table transports
- Employe -> table employes
- PosteEmploye -> table postes_employes
- StatutEmploye -> table statuts_employes
- Presence -> table presences
- SalaireEmploye -> table salaires_employes

9. Repositories JPA a creer

```
public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> { }
```

```
public interface RoleRepository extends JpaRepository<Role, Long> { }
```

```
public interface PermissionRepository extends JpaRepository<Permission, Long> { }
```

```
public interface StatutUtilisateurRepository extends JpaRepository<StatutUtilisateur, Long> { }
```

```
public interface LotPorcRepository extends JpaRepository<LotPorc, Long> { }
```

```
public interface RaceRepository extends JpaRepository<Race, Long> { }
```

```
public interface SexeRepository extends JpaRepository<Sexe, Long> { }
```

```
public interface StatutLotRepository extends JpaRepository<StatutLot, Long> { }

public interface MouvementLotPorcRepository extends JpaRepository<MouvementLotPorc, Long> { }

public interface TypeMouvementLotRepository extends JpaRepository<TypeMouvementLot, Long> { }

public interface PeseeLotRepository extends JpaRepository<PeseeLot, Long> { }

public interface ReproducteurRepository extends JpaRepository<Reproducteur, Long> { }

public interface StatutReproducteurRepository extends JpaRepository<StatutReproducteur, Long> { }

public interface CycleProductionRepository extends JpaRepository<CycleProduction, Long> { }

public interface EvenementReproductionRepository extends JpaRepository<EvenementReproduction, Long> { }

public interface TypeEvenementReproductionRepository extends JpaRepository<TypeEvenementReproduction, Long> { }

public interface MaladieRepository extends JpaRepository<Maladie, Long> { }

public interface TraitementRepository extends JpaRepository<Traitement, Long> { }

public interface SuiviSanitaireRepository extends JpaRepository<SuiviSanitaire, Long> { }

public interface StatutSuiviSanitaireRepository extends JpaRepository<StatutSuiviSanitaire, Long> { }

public interface VaccinRepository extends JpaRepository<Vaccin, Long> { }

public interface VaccinationRepository extends JpaRepository<Vaccination, Long> { }

public interface IngredientRepository extends JpaRepository<Ingredient, Long> { }

public interface TypeMouvementStockRepository extends JpaRepository<TypeMouvementStock, Long> { }

public interface MouvementStockAlimentRepository extends JpaRepository<MouvementStockAliment, Long> { }

public interface MelangeRepository extends JpaRepository<Melange, Long> { }

public interface MelangeIngredientRepository extends JpaRepository<MelangeIngredient, Long> { }

public interface RatioCroissanceRepository extends JpaRepository<RatioCroissance, Long> { }

public interface DistributionAlimentRepository extends JpaRepository<DistributionAliment, Long> { }

public interface ClientRepository extends JpaRepository<Client, Long> { }

public interface VenteRepository extends JpaRepository<Vente, Long> { }

public interface DetailVenteRepository extends JpaRepository<DetailVente, Long> { }

public interface PaiementRepository extends JpaRepository<Paiement, Long> { }

public interface FactureRepository extends JpaRepository<Facture, Long> { }

public interface CategorieDepenseRepository extends JpaRepository<CategorieDepense, Long> { }

public interface DepenseRepository extends JpaRepository<Depense, Long> { }

public interface TransportRepository extends JpaRepository<Transport, Long> { }

public interface EmployeRepository extends JpaRepository<Employe, Long> { }

public interface PosteEmployeRepository extends JpaRepository<PosteEmploye, Long> { }

public interface StatutEmployeRepository extends JpaRepository<StatutEmploye, Long> { }
```



```
public interface PresenceRepository extends JpaRepository<Presence, Long> { }

public interface SalaireEmployeeRepository extends JpaRepository<SalaireEmployee, Long> { }
```

10. Base de donnees complete

Types proposes: bigint pour les id Java Long, varchar pour texte court, text pour texte long, decimal pour montants/poids, date/datetime/time pour les dates, boolean pour actif/inactif. Si votre SGBD utilise integer pour les id, vous pouvez remplacer bigint par integer.

Table roles

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(100)	unique, not null
niveau_acces	integer	
description	text	
created_at	datetime	

Table permissions

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
code	varchar(100)	unique, not null
libelle	varchar(150)	not null
module	varchar(100)	

Table role_permissions

Colonne	Type	Contrainte / remarque
role_id	bigint	PK/FK roles.id
permission_id	bigint	PK/FK permissions.id

Table statuts_utilisateur

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table utilisateurs

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
nom	varchar(150)	not null
email	varchar(150)	unique, not null
mot_de_passe_hash	varchar(255)	not null
role_id	bigint	FK roles.id
statut_utilisateur_id	bigint	FK statuts_utilisateur.id
created_at	datetime	
updated_at	datetime	

Table races

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(100)	unique, not null
description	text	

Table sexes

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table statuts_lots

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table lots_porcs

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
code_lot	varchar(50)	unique, not null
type_entree	varchar(30)	Naissance/Achat
race_id	bigint	FK races.id
statut_lot_id	bigint	FK statuts_lots.id
nombre_initial	integer	not null
nombre_actuel	integer	not null
nombre_males_initial	integer	
nombre_femelles_initial	integer	
nombre_males_actuel	integer	
nombre_femelles_actuel	integer	
nombre_morts	integer	default 0
date_naissance_estimee	date	nullable
date_achat	date	nullable
prix_achat_total	decimal(12,2)	nullable
poids_moyen_initial_kg	decimal(10,2)	nullable
poids_moyen_actuel_kg	decimal(10,2)	nullable
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	
archived_at	datetime	nullable

Table type_mouvements_lots

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table mouvements_lots_porcs

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
lot_porc_id	bigint	FK lots_porcs.id
type_mouvement_lot_id	bigint	FK type_mouvements_lots.id
quantite	integer	not null
quantite_male	integer	nullable
quantite_femelle	integer	nullable
date_mouvement	datetime	not null
motif	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table pesees_lots

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
lot_porc_id	bigint	FK lots_porcs.id
poids_moyen_kg	decimal(10,2)	not null
date_pesee	date	not null
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table statuts_reproducteurs

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table reproducteurs

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
code_reproducteur	varchar(50)	unique, not null
nom	varchar(100)	
race_id	bigint	FK races.id
sexe_id	bigint	FK sexes.id
statut_reproducteur_id	bigint	FK statuts_reproducteurs.id
date_naissance	date	
date_arrivee	date	
prix_achat	decimal(12,2)	
poids_kg	decimal(10,2)	
nombre_parite	integer	nullable
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	
archived_at	datetime	nullable

Table type_evenements_reproduction

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table evenements_reproduction

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
femelle_id	bigint	FK reproducteurs.id
male_id	bigint	FK reproducteurs.id, nullable
type_evenement_reproduction_id	bigint	FK type_evenements_reproduction.id
lot_porc_id	bigint	FK lots_porcs.id, nullable
date_evenement	date	not null
nombre_porcelelets_nes	integer	
nombre_porcelelets_morts	integer	
nombre_porcelelets_vivants	integer	
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table cycles_production

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
code_cycle	varchar(50)	unique, not null
lot_porc_id	bigint	FK lots_porcs.id
date_debut	date	not null
date_fin_prevue	date	
date_fin_reelle	date	
nombre_naissances	integer	
nombre_pertes	integer	
nombre_vivants	integer	
nombre_vendables	integer	
statut_cycle	varchar(50)	
observation	text	
created_at	datetime	

Table maladies

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(150)	unique, not null
description	text	

Table traitements

Colonne	Type	Contrainte / remarque
---------	------	-----------------------

id	bigint	PK, auto-increment
maladie_id	bigint	FK maladies.id
libelle	varchar(150)	not null
prix	decimal(12,2)	
duree_guerison_jours	integer	
description	text	

Table statuts_suivi_sanitaire

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table suivis_sanitaires

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
lot_porc_id	bigint	FK lots_porcs.id, nullable
reproducteur_id	bigint	FK reproducteurs.id, nullable
nombre_porcs_malades	integer	nullable
maladie_id	bigint	FK maladies.id
traitement_id	bigint	FK traitements.id, nullable
statut_suivi_sanitaire_id	bigint	FK statuts_suivi_sanitaire.id
date_diagnostic	date	not null
date_guerison_prevue	date	
date_guerison_reelle	date	
symptomes	text	
diagnostic	text	
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table vaccins

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(150)	unique, not null
prix	decimal(12,2)	
delai_rappel_jours	integer	
description	text	
actif	boolean	default true

Table vaccinations

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
lot_porc_id	bigint	FK lots_porcs.id, nullable
reproducteur_id	bigint	FK reproducteurs.id, nullable
vaccin_id	bigint	FK vaccins.id
date_vaccination	date	not null
date_rappel	date	
dose	varchar(50)	
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table ingredients

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(150)	unique, not null
prix_kg	decimal(12,2)	not null
stock_actuel_kg	decimal(12,2)	default 0
seuil_min_kg	decimal(12,2)	default 0
unite	varchar(20)	default kg
actif	boolean	default true
created_at	datetime	

Table type_mouvements_stock

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table mouvements_stock_aliment

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
ingredient_id	bigint	FK ingredients.id
type_mouvement_stock_id	bigint	FK type_mouvements_stock.id
quantite_kg	decimal(12,2)	not null
prix_total	decimal(12,2)	
date_mouvement	datetime	not null
motif	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table melanges

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(150)	unique, not null
description	text	
cout_kg	decimal(12,2)	
statut	varchar(50)	
created_at	datetime	

Table melange_ingredients

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
melange_id	bigint	FK melanges.id
ingredient_id	bigint	FK ingredients.id
quantite_kg	decimal(12,2)	
pourcentage	decimal(5,2)	unique par melange+ingredient

Table ratios_croissance

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
race_id	bigint	FK races.id
melange_id	bigint	FK melanges.id
age_mois_min	integer	not null
age_mois_max	integer	not null
ration_kg_jour	decimal(10,2)	
gain_poids_estime_kg	decimal(10,2)	
created_at	datetime	

Table distributions_aliment

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
lot_porc_id	bigint	FK lots_porcs.id
melange_id	bigint	FK melanges.id
date_distribution	date	not null
quantite_kg	decimal(12,2)	not null
cout_total	decimal(12,2)	
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table clients

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
nom	varchar(150)	not null

type_client	varchar(50)	
contact	varchar(50)	
email	varchar(150)	
adresse	text	
created_at	datetime	
updated_at	datetime	

Table ventes

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
client_id	bigint	FK clients.id
date_vente	datetime	not null
montant_total	decimal(12,2)	not null
statut_vente	varchar(50)	Brouillon/Validee/Annulee
observation	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table details_vente

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
vente_id	bigint	FK ventes.id
lot_porc_id	bigint	FK lots_porcs.id
nombre_porcs_vendus	integer	not null
poids_total_kg	decimal(12,2)	
prix_kg	decimal(12,2)	
prix_unitaire	decimal(12,2)	
montant	decimal(12,2)	not null

Table paiements

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
vente_id	bigint	FK ventes.id
montant	decimal(12,2)	not null
mode_paiement	varchar(50)	
reference	varchar(100)	
date_paiement	datetime	not null

Table factures

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
vente_id	bigint	FK ventes.id, unique
numero_facture	varchar(100)	unique, not null
date_facture	datetime	not null
montant_total	decimal(12,2)	not null
fichier_url	varchar(255)	nullable

Table categories_depenses

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(100)	unique, not null

Table depenses

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
categorie_depense_id	bigint	FK categories_depenses.id
libelle	varchar(150)	not null
montant	decimal(12,2)	not null
date_depense	date	not null
description	text	
created_by	bigint	FK utilisateurs.id
created_at	datetime	

Table transports

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
depense_id	bigint	FK depenses.id, nullable
libelle	varchar(150)	
trajet	varchar(255)	
cout	decimal(12,2)	
date_transport	date	

Table postes_employes

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(100)	unique, not null

Table statuts_employes

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
libelle	varchar(50)	unique, not null

Table employes

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
nom	varchar(100)	not null
prenom	varchar(100)	
contact	varchar(50)	
adresse	text	
poste_employe_id	bigint	FK postes_employes.id
statut_employe_id	bigint	FK statuts_employes.id
date_embauche	date	
salaire_base	decimal(12,2)	
created_at	datetime	

Table presences

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
employe_id	bigint	FK employes.id
date_presence	date	not null
statut_presence	varchar(50)	Present/Absent/Retard
heure_arrivee	time	
heure_depart	time	
observation	text	
contrainte	unique(employe_id, date_presence)	

Table salaires_employes

Colonne	Type	Contrainte / remarque
id	bigint	PK, auto-increment
employe_id	bigint	FK employes.id
mois	integer	not null
annee	integer	not null
montant_base	decimal(12,2)	
prime	decimal(12,2)	
retenue	decimal(12,2)	
montant_net	decimal(12,2)	
statut_paiement	varchar(50)	En attente/Paye
date_paiement	date	
contrainte	unique(employe_id, mois, annee)	

11. Relations principales

- role_permissions.role_id -> roles.id
- role_permissions.permission_id -> permissions.id
- utilisateurs.role_id -> roles.id
- utilisateurs.statut_utilisateur_id -> statuts_utilisateur.id
- lots_porcs.race_id -> races.id
- lots_porcs.statut_lot_id -> statuts_lots.id
- lots_porcs.created_by -> utilisateurs.id
- mouvements_lots_porcs.lot_porc_id -> lots_porcs.id
- pesees_lots.lot_porc_id -> lots_porcs.id
- reproducteurs.race_id -> races.id
- reproducteurs.sexe_id -> sexes.id
- reproducteurs.statut_reproducteur_id -> statuts_reproducteurs.id
- evenements_reproduction.femelle_id/male_id -> reproducteurs.id
- cycles_production.lot_porc_id -> lots_porcs.id
- suivis_sanitaires.lot_porc_id -> lots_porcs.id
- suivis_sanitaires.reproducteur_id -> reproducteurs.id
- suivis_sanitaires.maladie_id -> maladies.id
- suivis_sanitaires.traitement_id -> traitements.id
- vaccinations.vaccin_id -> vaccins.id
- mouvements_stock_aliment.ingredient_id -> ingredients.id
- melange_ingredients.melange_id -> melanges.id
- melange_ingredients.ingredient_id -> ingredients.id
- distributions_aliment.lot_porc_id -> lots_porcs.id
- distributions_aliment.melange_id -> melanges.id
- ventes.client_id -> clients.id
- details_vente.vente_id -> ventes.id
- details_vente.lot_porc_id -> lots_porcs.id
- paiements.vente_id -> ventes.id
- factures.vente_id -> ventes.id
- depenses.categorie_depense_id -> categories_depenses.id
- employes.poste_employe_id -> postes_employes.id
- presences.employe_id -> employes.id
- salaires_employes.employe_id -> employes.id

12. Ordre conseille pour commencer le code

1. 1. Créer les entites JPA + repositories + donnees de reference: roles, statuts, races, sexes, types de mouvements.
2. 2. Créer authentication simple: LoginDTO, AuthController, AuthService, UtilisateurRepository.
3. 3. Créer le module lots: LotPorc, MouvementLotPorc, Peseelot.
4. 4. Créer le module reproduction: Reproducteur, CycleProduction, EvenementReproduction.
5. 5. Créer les modules commerce: Client, Vente, DetailVente, Paiement, Facture.
6. 6. Créer ressources: Ingredient, MouvementStock, Melange, Distribution.
7. 7. Créer sante: Vaccin, Vaccination, Maladie, Traitement, SuiviSanitaire.
8. 8. Créer personnel: Employe, Presence, Salaire.
9. 9. Créer dashboard et rapports en dernier, car ils consomment les donnees des autres modules.